

Lecture 4: R Basics Continued

Heather Mattie, PhD

September 11, 2025



Recipe of the Day!

Spiced Sweet Potato & Poblano Tostadas



Office Hours and Lab

Office Hours

Day	Time	Staff	Location
Monday	10-11am	Sajia	FXB G10
Tuesday	1-2pm	Heather	Building 1, 4 th floor, Room 421A
Wednesday	2:30-3:30pm	Carmen	Kresge 201 except on 10/1 where it will be held in Kresge 502
Thursday	12-1pm	Claire	Kresge 205

Lab

Day	Time	Location
Friday	11:30am-1pm	Zoom

***Lab will not be held every week**



R Basics

R Basics Roadmap

1. Data types
2. Vectors
3. Sorting
4. Indexing
5. Basic plots
6. Programming basics

R Basics

Indexing

Heather Mattie, PhD

murders dataset

```
head(murders)
```

```
##      state abb region population total
## 1  Alabama  AL  South    4779736    135
## 2   Alaska  AK   West     710231     19
## 3  Arizona  AZ   West    6392017    232
## 4 Arkansas  AR  South    2915918     93
## 5 California CA   West   37253956   1257
## 6  Colorado CO   West    5029196     65
```

max and which.max

The **max** function returns the maximum value in a vector

The **which.max** function returns the index of the maximum value

```
max(murders$total)
```

```
## [1] 1257
```

```
i_max <- which.max(murders$total)  
i_max
```

```
## [1] 5
```

```
murders$state[i_max]
```

```
## [1] "California"
```


Subsetting with logicals

Logical statements (**logicals**) can be used to index vectors

- Return **TRUE** or **FALSE** for each entry in a vector
- **TRUE** values are coded as 1
- **FALSE** values are coded as 0

```
murder_rate <- murders$total / murders$population * 100000  
ind <- murder_rate < 0.71  
ind
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE  
## [13] FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  
## [25] FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE TRUE FALSE  
## [37] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE  
## [49] FALSE FALSE FALSE
```

```
sum(ind)
```

```
## [1] 5
```

```
murders$state[ind]
```

```
## [1] "Hawaii"      "Iowa"         "New Hampshire" "North Dakota"  
## [5] "Vermont"
```

Logical operators

Logical operators are symbols or words used to connect two or more expressions

- Result in a **TRUE** or **FALSE**
- AND = **&**
- OR = **|**
- NOT = **!**

```
TRUE & TRUE
```

```
## [1] TRUE
```

```
TRUE & FALSE
```

```
## [1] FALSE
```

```
FALSE & FALSE
```

```
## [1] FALSE
```

```
west <- murders$region == "West"  
safe <- murder_rate <= 1  
ind <- safe & west  
murders$state[ind]
```

```
## [1] "Hawaii" "Idaho" "Oregon" "Utah" "Wyoming"
```

Logical operators

Logical operators are symbols or words used to connect two or more expressions

- Result in a **TRUE** or **FALSE**
- AND = **&**
- OR = **|**
- NOT = **!**

```
ind <- safe | west  
murders$state[ind]
```

```
## [1] "Alaska"      "Arizona"      "California"    "Colorado"  
## [5] "Hawaii"      "Idaho"        "Iowa"         "Maine"  
## [9] "Minnesota"   "Montana"      "Nevada"       "New Hampshire"  
## [13] "New Mexico"  "North Dakota" "Oregon"       "South Dakota"  
## [17] "Utah"        "Vermont"      "Washington"   "Wyoming"
```

```
ind <- safe & !west  
murders$state[ind]
```

```
## [1] "Iowa"        "Maine"        "Minnesota"    "New Hampshire"  
## [5] "North Dakota" "South Dakota" "Vermont"
```


which

The **which** function indicates which entries of a logical vector are **TRUE**

```
ind <- which(murders$state == "California")  
ind # this is the index that matches the California entry
```

```
## [1] 5
```

```
murder_rate[ind]
```

```
## [1] 3.374138
```


match

The **match** function indicates which indices of a second vector match each of the entries of a first vector

```
ind <- match(c("New York", "Florida", "Texas"), murders$state)
ind
```

```
## [1] 33 10 44
```

```
match(c("Boston", "Dakota", "Washington"), murders$state)
```

```
## [1] NA NA 48
```

```
murder_rate[ind]
```

`%in%`

The `%in%` function indicates whether or not each element in a vector is contained in another vector

```
c("Boston", "Dakota", "Washington") %in% murders$state
```

```
## [1] FALSE FALSE  TRUE
```

Summary

- Vectors can be subset with logical statements
- Logical operators (**&**, **|**, **!**) can connect two or more logical statements
- The **which**, **match** and **%in%** functions can be used to index where TRUE values are in a vector

R Basics

Basic Plots

Heather Mattie, PhD

Base R plots

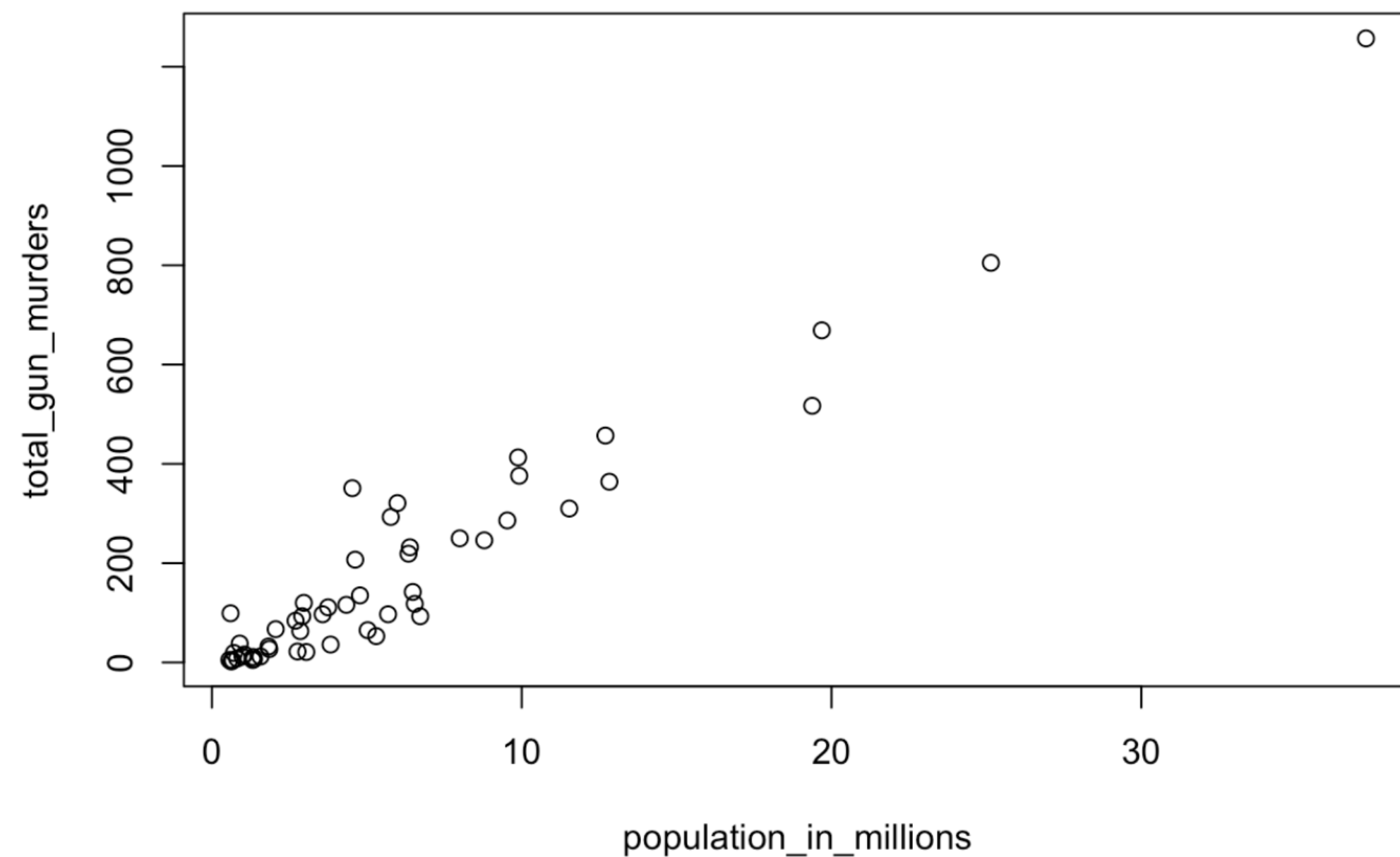
Plots can be effective and efficient ways to visualize data

- The type of plot depends on the **data type(s)**
- Exploratory data visualization is an important part of the data science project pipeline
- These plots don't require installing and loading any other R packages, but are rudimentary

Scatter plots

Scatter plots show the relationship between **two continuous** (numeric) variables

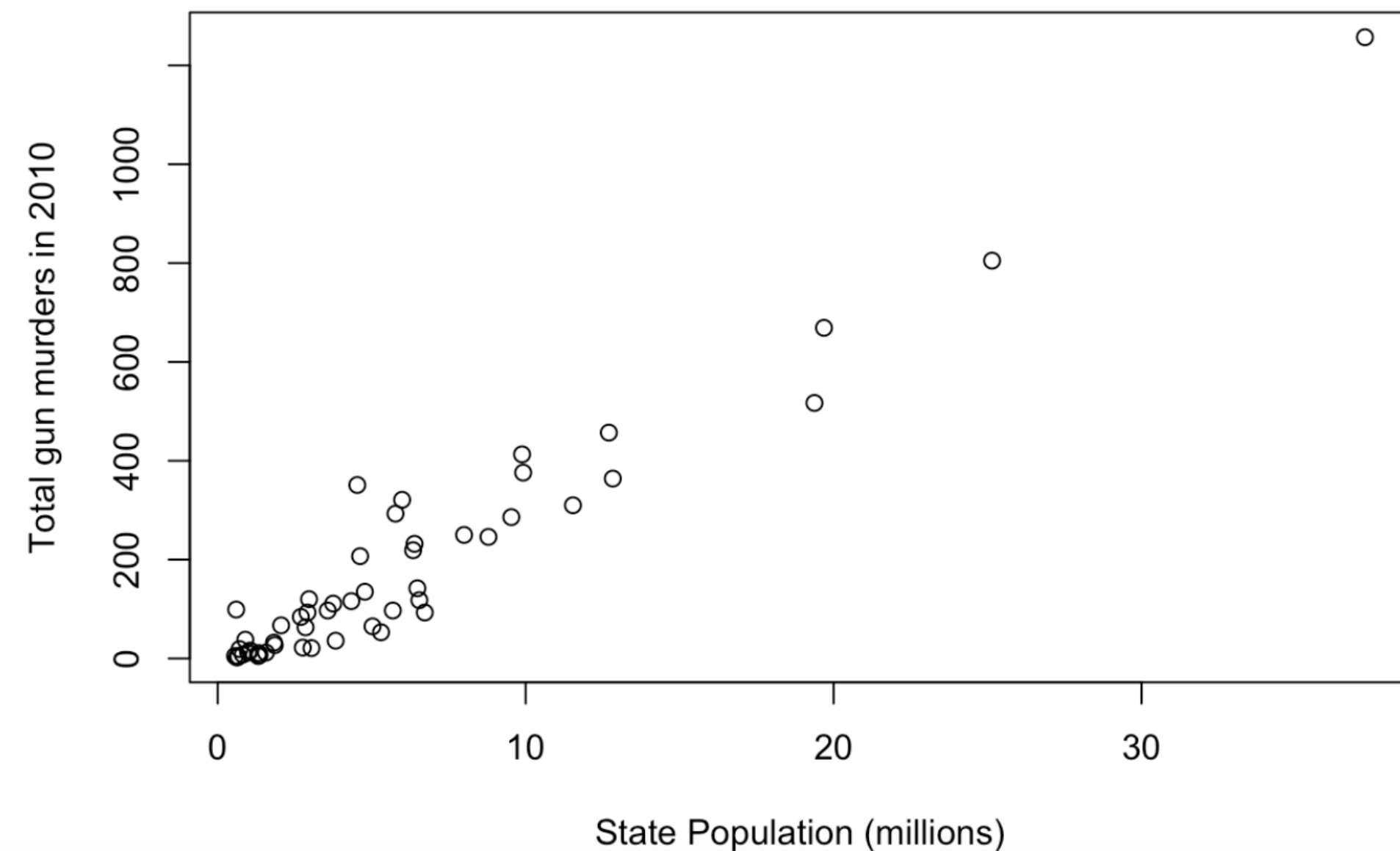
```
population_in_millions <- murders$population/10^6  
total_gun_murders <- murders$total  
plot(x = population_in_millions, y = total_gun_murders)
```



Scatter plots

Scatter plots show the relationship between two continuous (numeric) variables

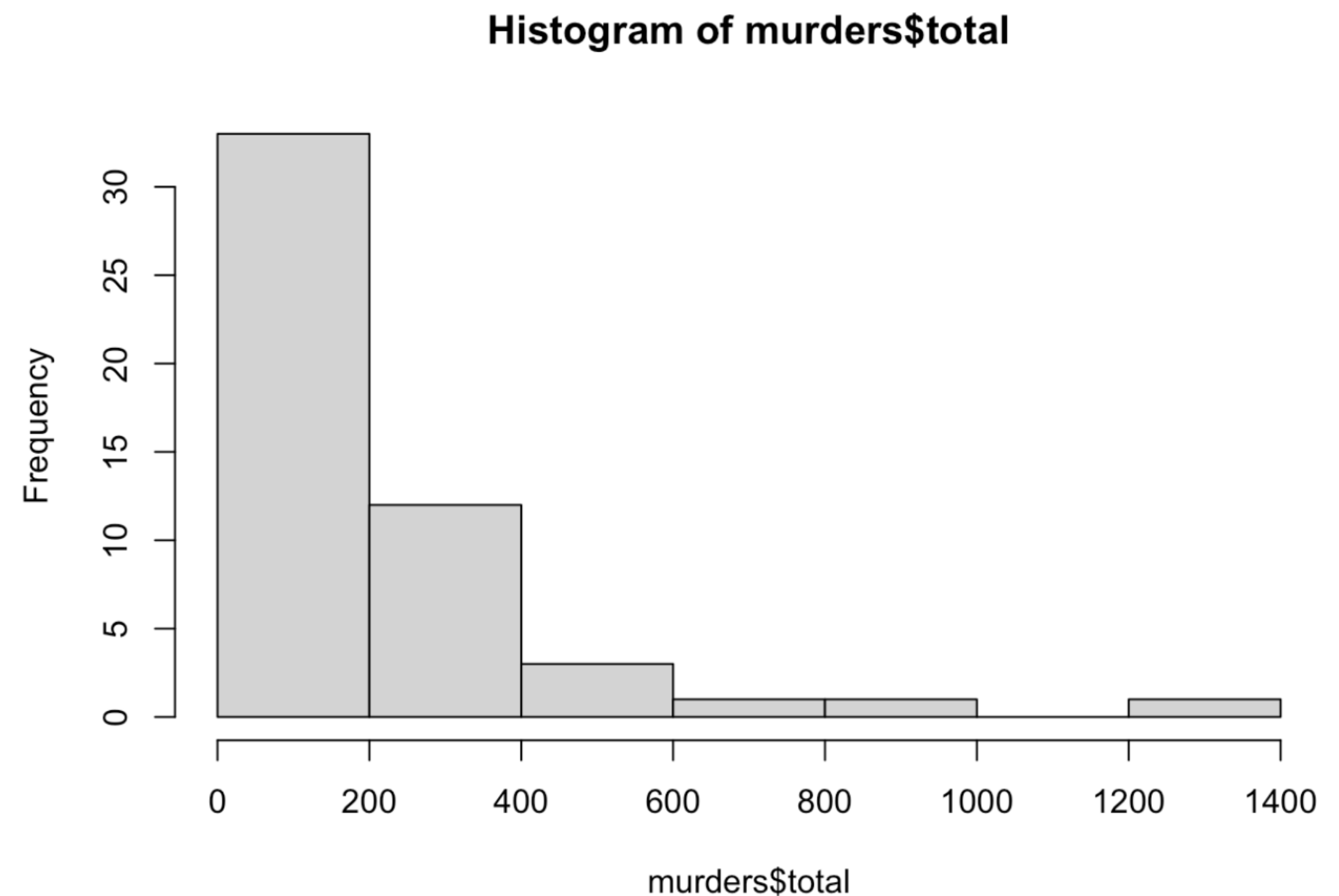
```
population_in_millions <- murders$population/10^6
total_gun_murders <- murders$total
plot(x = population_in_millions, y = total_gun_murders,
     xlab = "State Population (millions)",
     ylab = "Total gun murders in 2010")
```



Histograms

Histograms plot the distribution of values for **one continuous** (numeric) variable

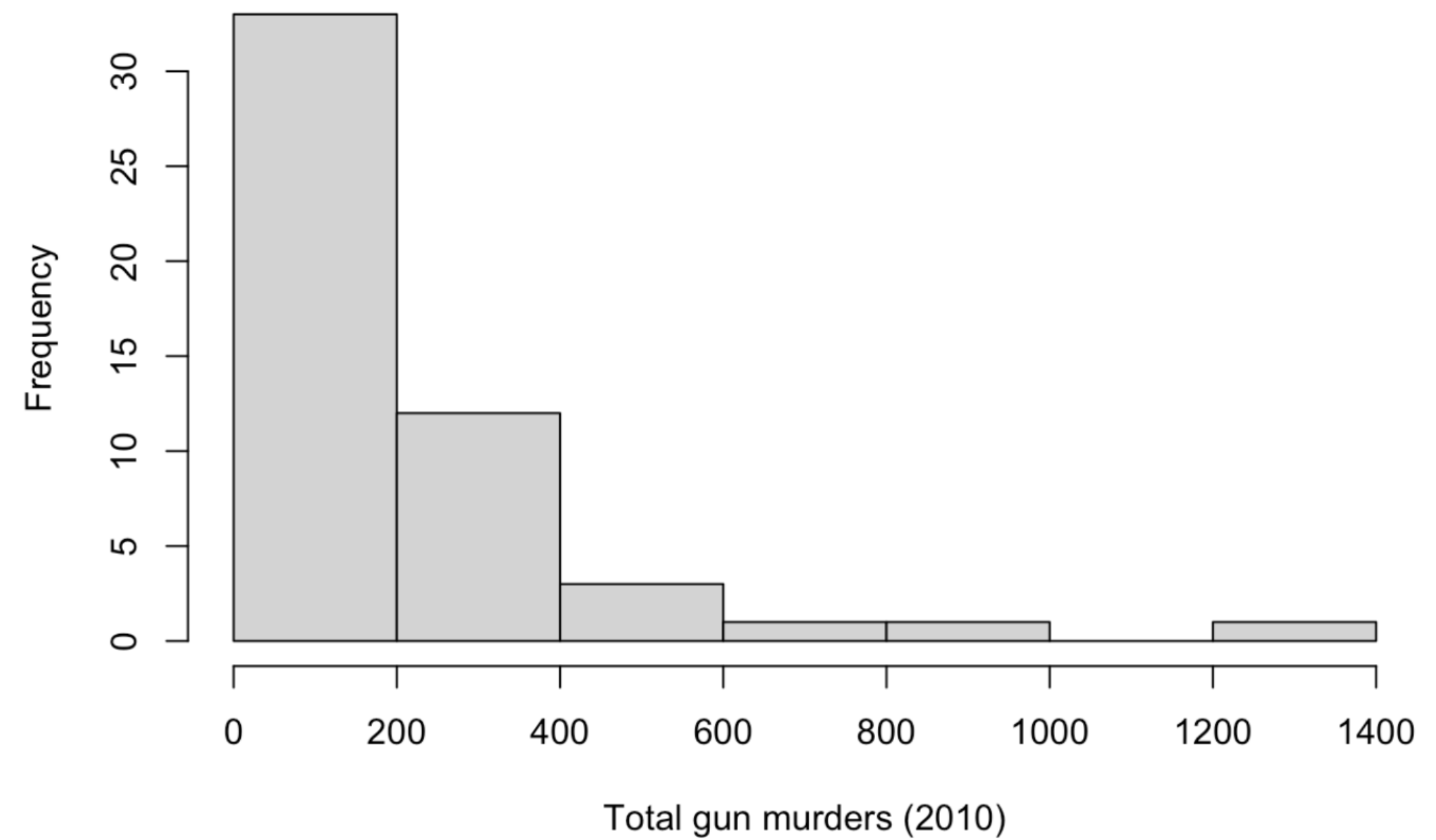
```
hist(murders$total)
```



Histograms

Histograms plot the distribution of values for **one continuous** (numeric) variable

```
hist(murders$total, xlab = "Total gun murders (2010)",  
     main = "")
```

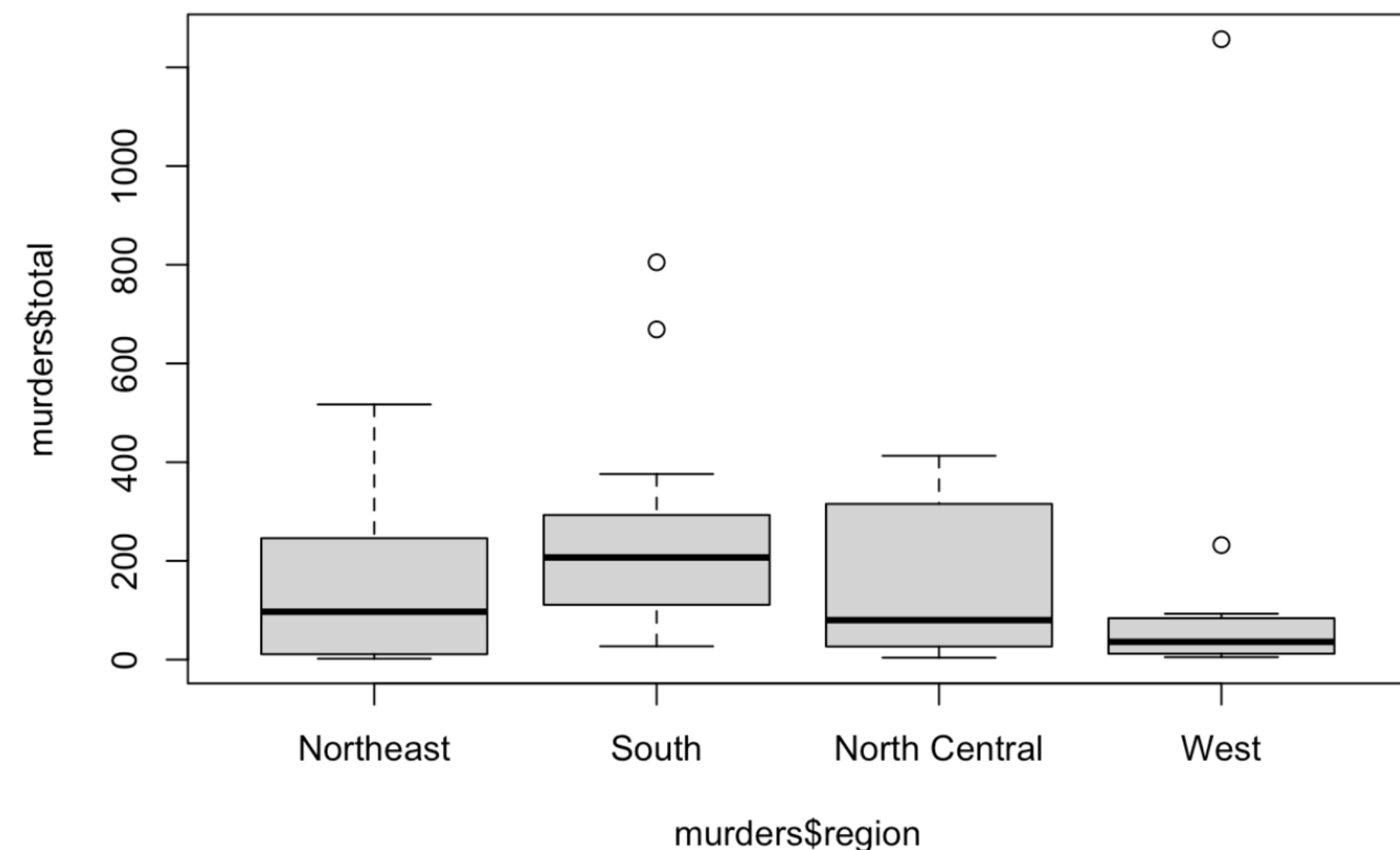


Boxplots

Boxplots display the distribution of a **continuous** variable based on a five-number summary

- Minimum, first quartile (Q1), median, third quartile (Q3), and maximum
- Display outliers if they exist
- Can plot multiple boxplots in one plot

```
boxplot(murders$total ~ murders$region)
```

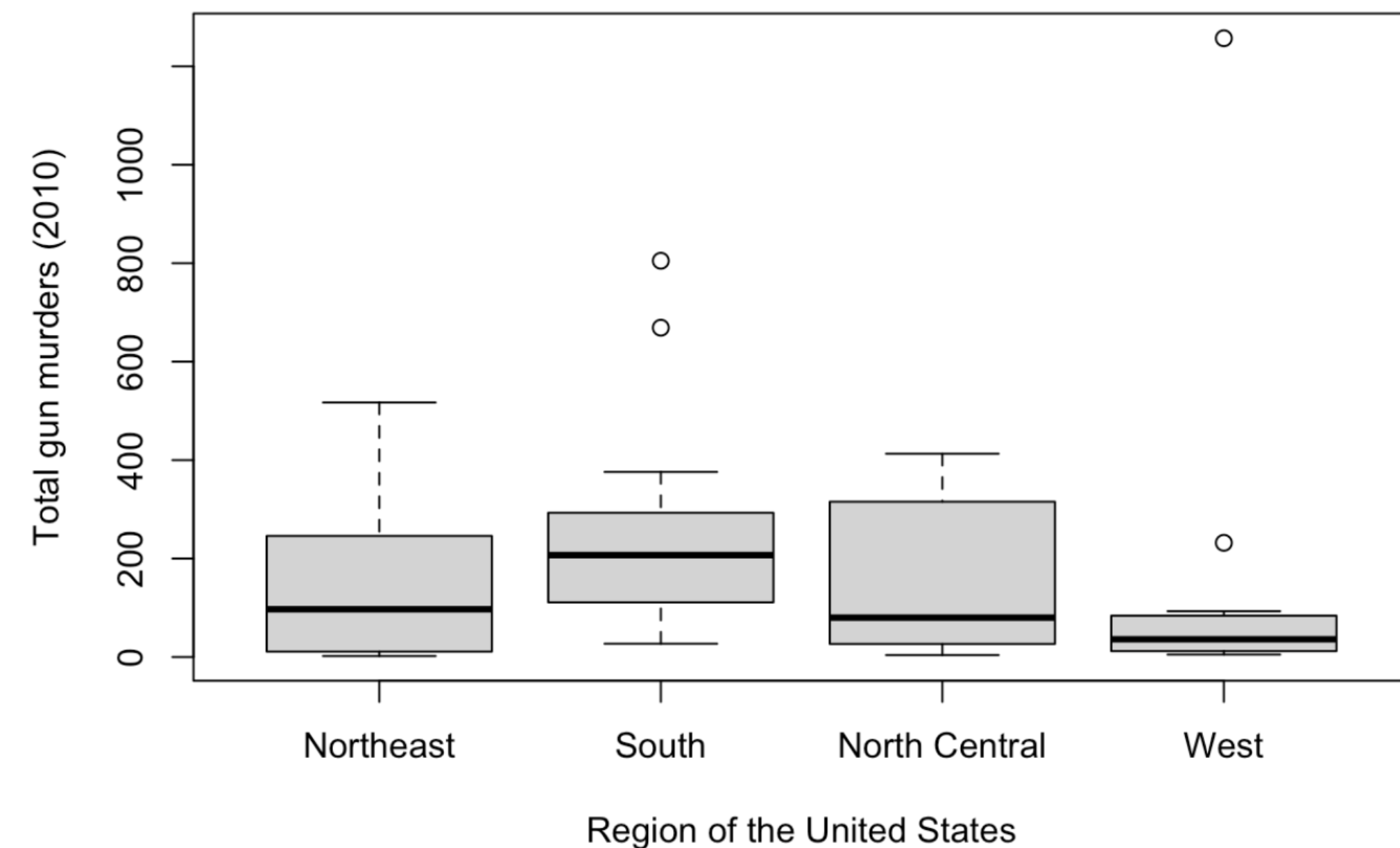


Boxplots

Boxplots display the distribution of a **continuous** variable based on a five-number summary

- Minimum, first quartile (Q1), median, third quartile (Q3), and maximum
- Display outliers if they exist
- Can plot multiple boxplots in one plot

```
boxplot(murders$total ~ murders$region,  
        xlab = "Region of the United States",  
        ylab = "Total gun murders (2010)")
```

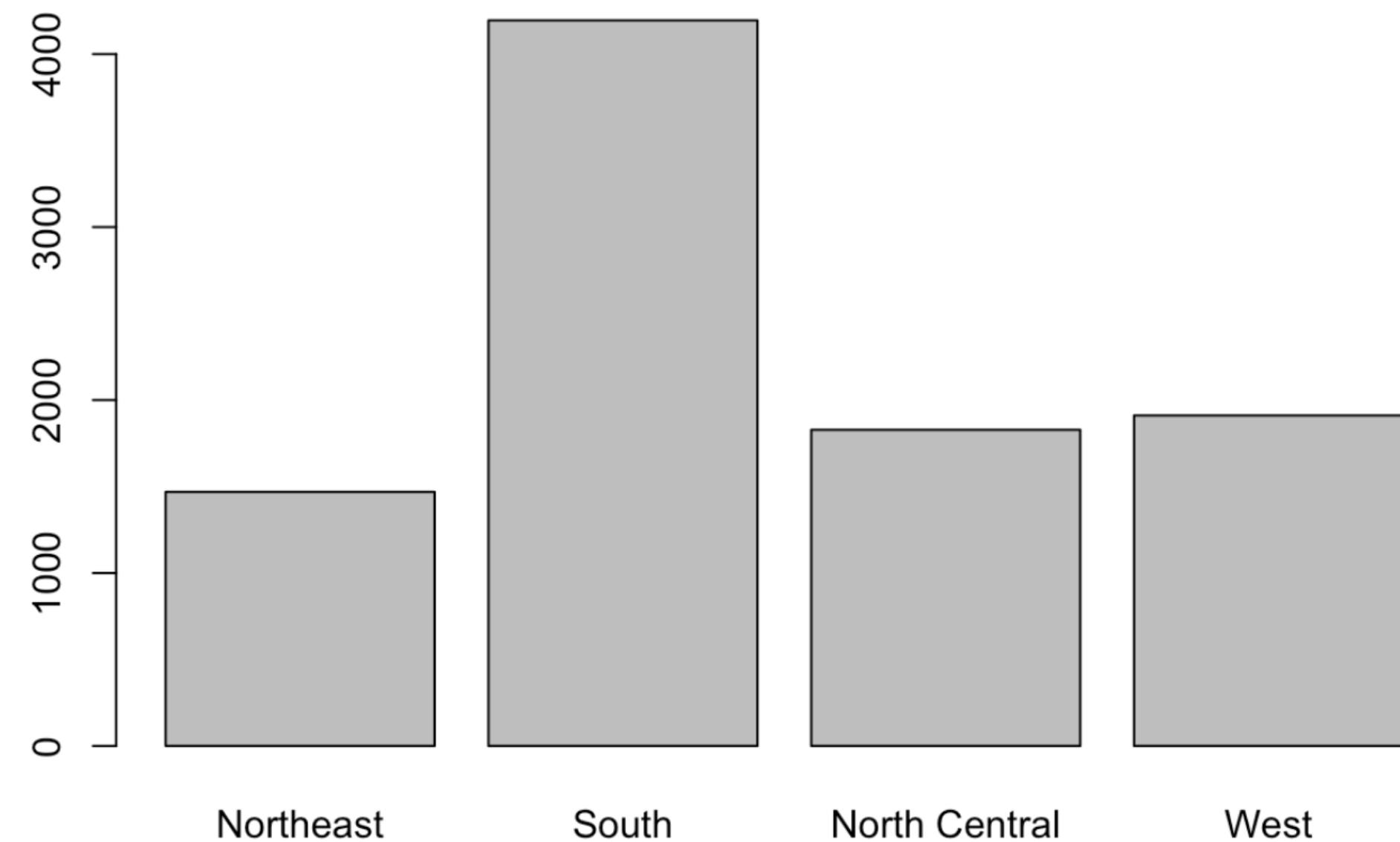


Barplots

Barplots show the relationship between a **numeric** variable and **categorical** variable

- Each bar represents a category
- The length of a bar represents its numeric value

```
data <- data.frame(  
  name = c("Northeast", "South", "North Central", "West"),  
  value = c(1469, 4195, 1828, 1911))  
barplot(height = data$value, names = data$name)
```

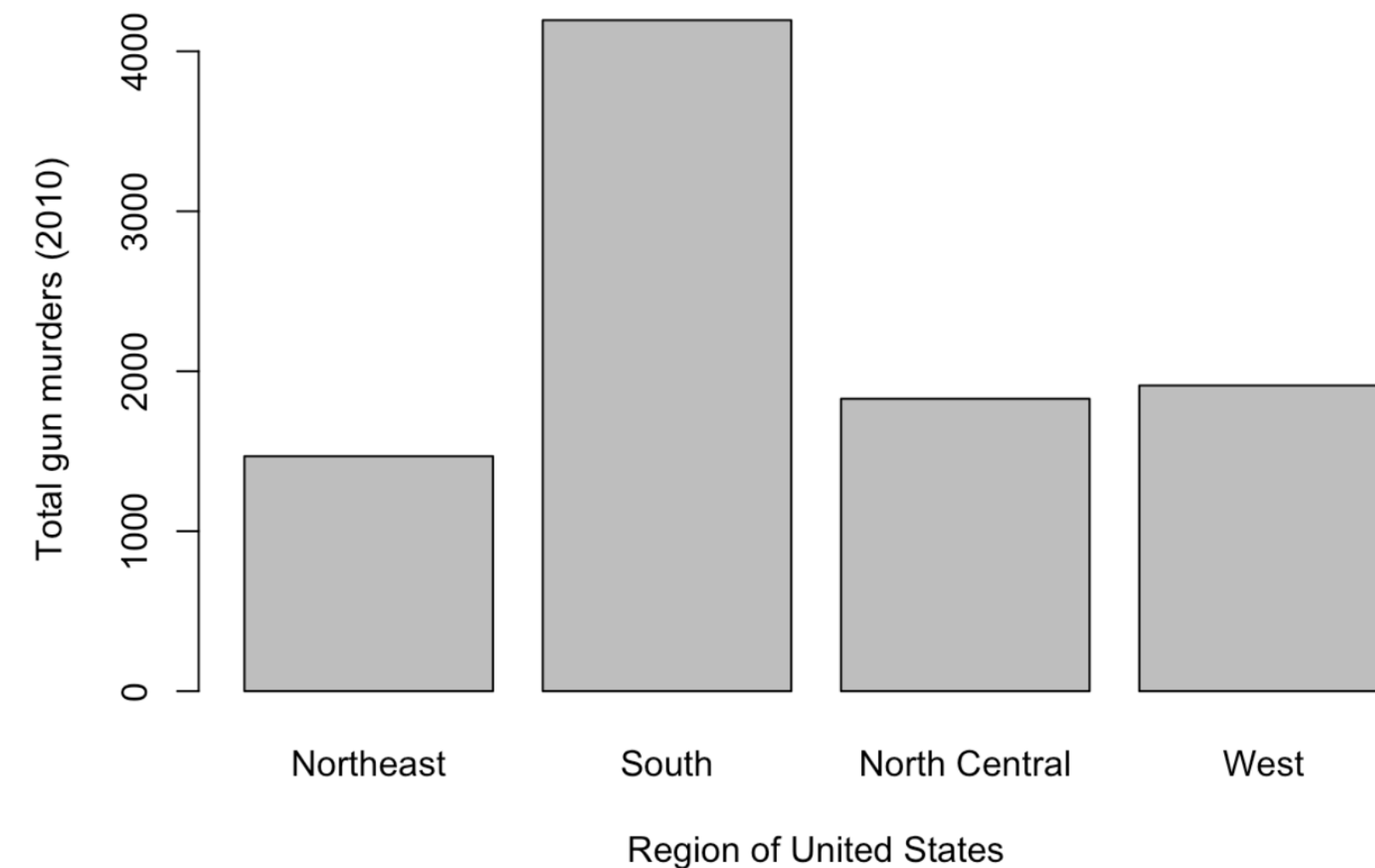


Barplots

Barplots show the relationship between a **numeric** variable and **categorical** variable

- Each bar represents a category
- The length of a bar represents its numeric value

```
data <- data.frame(  
  name = c("Northeast", "South", "North Central", "West"),  
  value = c(1469, 4195, 1828, 1911))  
barplot(height = data$value, names = data$name,  
        xlab = "Region of United States",  
        ylab = "Total gun murders (2010)")
```



Summary

- Plots are a necessary and effective tool to gain insights into data
- Base R plot functions can be used to create rudimentary plots
- Scatter plots, histograms, boxplots, and barplots are commonly used to display data

R Basics

Programming Basics

Heather Mattie, PhD

Programming basics

Programming is the process of creating a set of instructions (code) that a computer can understand and execute to perform tasks or solve problems

- Conditional expressions
- For loops
- Defining functions

Conditional expressions

Conditional expressions (if-else statements) execute different blocks of code based on whether a condition is true or false

```
a <- 0

if(a != 0){
  print(1/a)
} else{
  print("No reciprocal for 0.")
}
```

```
## [1] "No reciprocal for 0."
```

```
a <- 2

if(a != 0){
  print(1/a)
} else{
  print("No reciprocal for 0.")
}
```

```
## [1] 0.5
```


Conditional expressions

Conditional expressions (if-else statements) execute different blocks of code based on whether a condition is true or false

```
a <- 0  
ifelse(a > 0, 1/a, NA)
```

```
## [1] NA
```

```
a <- c(0, 1, 2, -4, 5)  
result <- ifelse(a > 0, 1/a, NA)  
result
```

```
## [1] NA 1.0 0.5 NA 0.2
```

Conditional expressions

Conditional expressions (if-else statements) execute different blocks of code based on whether a condition is true or false

```
library(dslabs)
data(murders)
murder_rate <- murders$total/murders$population*100000
ind <- which.min(murder_rate)

ifelse(murder_rate[ind] < 0.5, murders$state[ind], print
("Minumum murder rate is not that low.))
```

```
## [1] "Vermont"
```

```
ifelse(murder_rate[ind] < 0.25, murders$state[ind], print
("Minumum murder rate is not that low.))
```

```
## [1] "Minumum murder rate is not that low."
```

For loops

For loops repeatedly execute a block of code a specific number of times or until a certain condition is met

- Used for iteration
- Used when number of repetitions is known in advance

```
for(i in 1:5){  
  print(i)  
}
```

```
## [1] 1  
## [1] 2  
## [1] 3  
## [1] 4  
## [1] 5
```


For loops

```
# Filter to Northeast region
northeast_data <- murders[murders$region == "Northeast", ]

# Use a for loop to print murder rates
for (i in 1:nrow(northeast_data)) {
  state_name <- northeast_data$state[i]
  rate <- (northeast_data$total[i] / northeast_data$population[i]) * 100000
  cat(state_name, "has a murder rate of", round(rate, 2), "per 100,000 people.\n")
}
```

```
## Connecticut has a murder rate of 2.71 per 100,000 people.
## Maine has a murder rate of 0.83 per 100,000 people.
## Massachusetts has a murder rate of 1.8 per 100,000 people.
## New Hampshire has a murder rate of 0.38 per 100,000 people.
## New Jersey has a murder rate of 2.8 per 100,000 people.
## New York has a murder rate of 2.67 per 100,000 people.
## Pennsylvania has a murder rate of 3.6 per 100,000 people.
## Rhode Island has a murder rate of 1.52 per 100,000 people.
## Vermont has a murder rate of 0.32 per 100,000 people.
```

Defining functions

Custom **functions** can be created and used in R

- If no such function exists
- If your code performs the same task more than once

```
# Anatomy of a function
```

```
name <- function(x){  
  code to run  
  output  
}
```

```
avg <- function(x){  
  s <- sum(x)  
  n <- length(x)  
  s/n  
}
```

```
x <- 1:100  
avg(x)
```

```
## [1] 50.5
```


Defining functions

Custom **functions** can be created and used in R

- If no such function exists
- If your code performs the same task more than once

```
compute_murder_rate <- function(total, population) {  
  rate <- (total / population) * 100000  
  return(rate)  
}  
  
# Get total murders and population for New York  
ny_total <- murders$total[murders$state == "New York"]  
ny_pop <- murders$population[murders$state == "New York"]  
  
# Compute rate  
compute_murder_rate(ny_total, ny_pop)
```

```
## [1] 2.66796
```


Summary

- If-else statements can be used to execute different parts of code based on a condition
- For loops repeatedly execute code for a set number of times
- Custom functions can be created and used in R